

Week-1

Graded Assignment

Question 1

The function `ls()` in R will

- set a new working directory path
- ✓ list all objects in our working environment
- display the path to our working directory
- list all files in our working directory

Solution:

The function `ls()` is used in R to display the names of objects that are currently present in the working environment.

For example, suppose we create two variables:

```
x <- 10
name <- "Archit"
```

If we execute

```
ls()
```

R displays the objects available in the current environment, such as `x` and `name`.

It is important not to confuse `ls()` with other R functions:

- `setwd()` changes the working directory.
- `getwd()` displays the current working directory.
- `list.files()` lists files in the working directory.
- `ls()` lists objects in the R environment.

Therefore, the correct answer is:

list all objects in our working environment

Hence, option (B) is correct.

Question 2

Consider the following code:

```
room.temperature <- as.integer(
  readline(prompt="Enter room temperature in degree celsius: ")
)
```

```

if (room.temperature < 19) {
    print("The room is chilly. Please turn the heater on!")
} else {
    if (room.temperature >= 19 && room.temperature <= 25) {
        print("The room is warm")
        print("Enjoy the stay!")
    } else {
        print("The room is hot")
        print("Please turn the air conditioning on!")
    }
}

```

If a room temperature of 20 degree Celsius is input to the above code, choose the output displayed.

- "The room is chilly. Please turn the heater on!"
- ✓ "The room is warm" "Enjoy the stay!"
- "The room is hot" "Please turn the air conditioning on!"
- None of the above

Solution:

The input is

$$\text{room.temperature} = 20.$$

The first condition is

$$20 < 19.$$

This condition is **FALSE**, so the first **if** block is not executed.

R therefore moves to the **else** block. Inside it, we check

$$20 \geq 19 \quad \text{and} \quad 20 \leq 25.$$

Both conditions are true:

$$20 \geq 19 = \text{TRUE}$$

and

$$20 \leq 25 = \text{TRUE}.$$

Therefore,

$$\text{TRUE} \ \&\& \ \text{TRUE} = \text{TRUE}.$$

Hence, the inner **if** block is executed:

```

print("The room is warm")

```

```
print("Enjoy the stay!")
```

Thus, the displayed output is:

"The room is warm" "Enjoy the stay!"

Hence, option (B) is correct.

Question 3

Consider the code below to create a dataframe city_data:

```
name <- c("City A", "City B", "City C")
weather <- c("Sunny", "cloudy", "Rainy")
city_data <- data.frame(name, weather)
```

Choose the correct answer based on the output expected on running the below code.

```
city_data$weather[city_data$name == "City C"] <- "Snowy"
print(city_data)
```

- ✓ In the column weather in dataframe city_data, “Rain” is replaced by “Snowy”
- In the column name in dataframe city_data, “City C” is replaced by “Snowy”
- No change occurs to the entries in the dataframe city_data
- The code raises an error

Solution:

Initially, the dataframe is:

name	weather
City A	Sunny
City B	cloudy
City C	Rainy

The important statement is:

```
city_data$weather[city_data$name == "City C"] <- "Snowy"
```

Let us break this statement into parts.

First,

```
city_data$name == "City C"
```

checks which row has the name "City C".

The resulting logical vector is:

(FALSE, FALSE, TRUE).

Next,

```
city_data$weather[city_data$name == "City C"]
```

selects the weather value corresponding to City C. That value is initially "Rainy".

The assignment

```
<- "Snowy"
```

therefore replaces only that value.

The resulting dataframe is:

name	weather
City A	Sunny
City B	cloudy
City C	Snowy

Therefore, "Rainy" is replaced by "Snowy" in the `weather` column.

Hence, option (A) is correct.

Question 4

Consider the code given below.

```
city_weather = list(  
  "City A",  
  "City B",  
  "City C",  
  c("Sunny", "cloudy", "Rainy")  
)
```

Choose the correct command to access the variable "Cloudy".

- ✓ `city_weather[[4]][2]`
 - `city_weather[[5]]`
 - `city_weather[[2]][2]`
 - None of the above
-

Solution:

The list `city_weather` contains four elements:

Position	Value
1	"City A"
2	"City B"
3	"City C"
4	<code>c("Sunny", "cloudy", "Rainy")</code>

The fourth element is itself a vector:

```
c("Sunny", "cloudy", "Rainy").
```

We use

```
city_weather[[4]]
```

to extract the fourth element of the list.

This gives:

```
c("Sunny", "cloudy", "Rainy").
```

Now we need the second element of this vector. Therefore, we use

```
city_weather[[4]][2]
```

which gives

"cloudy"

.

Thus, the correct command is:

city_weather[[4]][2]

Hence, option (A) is correct.

Question 5

Consider the following code.

```
A = matrix(c(1:42), nrow = 6, ncol = 7, byrow = T)
B = A[-2,]
```

Based on the output of the above code, choose the correct option.

- ☐ B is a matrix consisting of only elements of the second row of matrix A
- ☐ B is a matrix consisting of same elements as that of A
- ☐ The code raises an error
- ☒ B is a matrix consisting of all elements of A excluding the second row

Solution:

The matrix is created using:

```
A = matrix(c(1:42), nrow = 6, ncol = 7, byrow = T)
```

Since there are 42 elements and the matrix has 6 rows and 7 columns, the matrix is:

$$A = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 \\ 8 & 9 & 10 & 11 & 12 & 13 & 14 \\ 15 & 16 & 17 & 18 & 19 & 20 & 21 \\ 22 & 23 & 24 & 25 & 26 & 27 & 28 \\ 29 & 30 & 31 & 32 & 33 & 34 & 35 \\ 36 & 37 & 38 & 39 & 40 & 41 & 42 \end{bmatrix}.$$

The next statement is:

```
B = A[-2,]
```

In R, a negative index means that the corresponding row or column should be excluded.
Here, -2 means:

exclude row 2.

The comma after -2 indicates that all columns should be retained.
Therefore, B contains rows

1, 3, 4, 5, 6.

Hence,

$$B = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 \\ 15 & 16 & 17 & 18 & 19 & 20 & 21 \\ 22 & 23 & 24 & 25 & 26 & 27 & 28 \\ 29 & 30 & 31 & 32 & 33 & 34 & 35 \\ 36 & 37 & 38 & 39 & 40 & 41 & 42 \end{bmatrix}.$$

Thus, B consists of all elements of A except the second row.
Therefore, option (D) is correct.

Question 6

The parameter “collapse” in the function paste()

- adjusts the display of the string to left, right or center
- add space in between two strings
- ✓ eliminates the space in between two strings
- eliminates the space within two words in a string

Solution:

The `paste()` function is used to combine strings.

The `collapse` parameter specifies the separator that should be used when multiple elements are combined into a single string.

For example,

```
paste(c("Hello", "World"), collapse = " ")
```

produces:

"Hello World".

Here, the value of `collapse` is a space:

`collapse = " "`.

If we use an empty string,

```
paste(c("Hello", "World"), collapse = "")
```

the two elements are joined without a space:

"HelloWorld".

Thus, when `collapse=""` is used, the space/separator between the two strings is eliminated.

Therefore, according to the given question, the correct answer is:

eliminates the space in between two strings

Hence, option (C) is correct.

Question 7

Consider the following code in R.

```
find.function <- function(a) {  
  for(i in 1:a) {  
    b <- i^2  
    print(b)  
  }  
}
```

If we supply 4 as an argument to the function `find.function`, after executing the above code, the result displayed would be

- ☐ the numbers "1,2,3,4"
- ☒ the numbers "1,4,9,16"
- ☐ the number "4"
- ☐ None of the above

Solution:

The function is:

```
find.function <- function(a) {  
  for(i in 1:a) {  
    b <- i^2  
    print(b)  
  }  
}
```

We supply

$a = 4$.

Therefore, the loop

```
for(i in 1:a)
```

becomes

$$i = 1, 2, 3, 4.$$

For every value of i , the statement

```
b <- i^2
```

calculates the square of i .

The values are:

i	i^2
1	1
2	4
3	9
4	16

The `print(b)` statement therefore displays:

1, 4, 9, 16.

Hence, the correct answer is:

the numbers "1,4,9,16"

Therefore, option (B) is correct.

Question 8

The operator `%in%` is used for

- multiplication of a matrix with its transpose
- generating a sequence of numbers in a vector
- ✓ identifying if an element belongs to a vector
- none of the above

Solution:

The operator `%in%` is the membership operator in R.

It checks whether an element is present in a vector.

For example,

```
3 %in% c(1, 2, 3, 4)
```

returns:

TRUE

because 3 is present in the vector.

Similarly,

```
7 %in% c(1, 2, 3, 4)
```

returns:

FALSE

because 7 is not present in the vector.

Therefore,

<code>%in%</code> identifies if an element belongs to a vector
--

Hence, option (C) is correct.

Question 9

The output of compiling and executing the following code in R would be

```
v_name <- c("Green", "apple")
cnt <- 2
```

```
repeat {
  print(v_name)
  cnt <- cnt + 1

  if(cnt > 5) {
    break
  }
}
```

- The term “Green” “apple” printed 5 times.
- The term “Green apple” printed 3 times.
- The term “Green” printed 4 times
- ✓ The term “Green” “apple” printed 4 times.

Solution:

The vector is initialized as:

```
v_name <- c("Green", "apple")
```

Therefore, every execution of

```
print(v_name)
```

prints the two elements:

"Green" and "apple".

The counter starts with

$$cnt = 2.$$

The **repeat** loop first prints the vector and then executes

```
cnt <- cnt + 1
```

After each iteration, the value of **cnt** is:

Iteration	cnt before increment	cnt after increment
1	2	3
2	3	4
3	4	5
4	5	6

After the fourth iteration,

$$cnt = 6.$$

The condition

```
if(cnt > 5)
```

becomes

$$6 > 5,$$

which is **TRUE**. Therefore, **break** terminates the loop.

Hence, **print(v.name)** executes exactly four times.

Therefore,

The term “Green” “apple” is printed 4 times.

Hence, option (D) is correct.

Question 10

The “**next**” statement in R programming is useful

- ✓ for skipping the current iteration of a loop without terminating it
- for terminating the current iteration of a loop
- for evaluating the current iteration of a loop without terminating it
- None of the above

Solution:

The **next** statement is used inside loops.

When R encounters **next**, it skips the remaining statements in the current iteration and

immediately proceeds to the next iteration of the loop.

For example:

```
for(i in 1:5) {  
    if(i == 3) {  
        next  
    }  
    print(i)  
}
```

The loop starts with

$$i = 1.$$

It prints 1.

For $i = 2$, it prints 2.

When

$$i = 3,$$

the condition

```
i == 3
```

is true. Therefore, `next` is executed. The remaining statements in that iteration are skipped, so 3 is not printed.

The loop then continues with $i = 4$ and $i = 5$.

The output is therefore:

1, 2, 4, 5.

It is important to distinguish `next` from `break`:

- `next`: skips the current iteration and continues the loop.
- `break`: terminates the entire loop.

Therefore, the correct answer is:

for skipping the current iteration of a loop without terminating it

Hence, option (A) is correct.
